MODULE 18 · DOMAIN 2 - CONTAINER ORCHESTRATION

Pod security

Pod Security Standards and Admission, securityContext, and admission policy engines.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 18 of 35

admin@sthithapragna.com

RBAC decides who may create a Pod. Pod security decides **what that Pod may do to the node**.

The settings that make a Pod dangerous

- **privileged: true** gives the container almost all of the host's power.
- **hostNetwork, hostPID, hostIPC** share the node's own namespaces.
- **hostPath volumes** mount the node's filesystem into the container.
- **Running as root, extra capabilities,** and allowing privilege escalation.

> ON THE EXAM
>
> Each of these turns a container escape into a node compromise. Pod security blocks them.

The three Pod Security Standards

LEVEL	WHAT IT ALLOWS	FOR
Privileged	Everything. No restrictions at all	Trusted system components: CNI, CSI, monitoring agents
Baseline	Blocks known escalations: privileged, host namespaces, hostPath, dangerous capabilities	Most ordinary applications, with no changes
Restricted	Baseline, plus non-root, no privilege escalation, all capabilities dropped, seccomp	Hardened, security-critical workloads

ON THE EXAM

Privileged, Baseline, Restricted: from anything goes, to safe by default, to hardened.

Currency check: PodSecurityPolicy is gone

- **PodSecurityPolicy was removed in Kubernetes 1.25.**
- **Pod Security Admission** replaced it: built in, and stable since 1.25.
- **Simpler:** no policy objects to write; a namespace label picks a standard.
- Older material describing PSP objects and their bindings is out of date.

> ON THE EXAM
>
> Asked what replaced PodSecurityPolicy: Pod Security Admission, applying the Pod Security Standards.

Pod Security Admission modes

MODE	WHAT HAPPENS TO A VIOLATING POD
`enforce`	Rejected. It is never created
`audit`	Allowed, but recorded in the audit log
`warn`	Allowed, but the user sees a warning

ON THE EXAM

Each mode can use a different level. Enforce what you are sure of; warn and audit what you are testing.

Setting the standard for a namespace

```
1 apiVersion: v1
2 kind: Namespace
3 metadata:
4   name: quayside
5   labels:
6     pod-security.kubernetes.io/enforce: baseline
7     pod-security.kubernetes.io/warn: restricted
8     pod-security.kubernetes.io/audit: restricted
```

6 Reject anything below Baseline, from now on.

7-8 Report what Restricted would reject, without blocking it.

THE TAKEAWAY

Labels can also pin a version, such as enforce-version: v1.37, so upgrades do not tighten rules by surprise.

The gotcha: enforce checks Pods, not Deployments

- **A Deployment that breaks the standard is accepted.** It is not a Pod.
- **Its Pods are then rejected,** so the Deployment shows zero ready replicas.
- **The reason is in the ReplicaSet's events:** FailedCreate, violates PodSecurity.
- **warn and audit do check workload templates,** so you see warnings at apply time.

> ON THE EXAM
>
> Deployment accepted, no Pods, in a namespace with enforce: check the ReplicaSet events.

A Pod that meets Restricted

```
1  spec:
2    securityContext:
3      runAsNonRoot: true
4      seccompProfile: {type: RuntimeDefault}
5    containers:
6    - name: api
7      image: quayside/booking-api:2.4.1
8      securityContext:
9        allowPrivilegeEscalation: false
10       readOnlyRootFilesystem: true
11       capabilities: {drop: [ALL]}
```

3-4 Pod level: never root, and the runtime's default seccomp filter.

9-11 Container level: no escalation, no capabilities. Read-only root is extra hardening.

securityContext settings

SETTING	EFFECT
`runAsNonRoot, runAsUser`	Refuse root; or run as a specific user ID
`allowPrivilegeEscalation: false`	Block setuid tricks that gain privileges
`capabilities: drop / add`	Remove or grant specific Linux capabilities
`privileged`	Near-full host power. Avoid
`readOnlyRootFilesystem`	The container cannot modify its own files
`seccompProfile`	Filter which system calls the process may make

ON THE EXAM

Pod-level settings apply to every container; container-level settings override them.

Currency check: user namespaces

- **hostUsers: false** gives a Pod its own user namespace.
- **Root inside maps to an unprivileged user on the host,** so a breakout is not root on the node.
- **Stable since Kubernetes 1.36,** April 2026. Linux only.
- **Needs a recent kernel and runtime,** and supporting volume types.

> ON THE EXAM
>
> Isolation stronger than runAsNonRoot alone, without the cost of a sandboxed runtime.

Admission controllers are the last check before storage: they can **change** a request, or **reject** it.

Where admission fits

01

Authenticated, authorised

The request passed module 17's checks.

02

Mutating

Defaults and changes: built-ins, webhooks, policies.

03

Schema check

Is the object well formed?

04

Validating

Accept or reject: PodSecurity, quotas, webhooks, policies.

05

Stored

Written to etcd.

ON THE EXAM

Mutating runs first, so validating sees the final object.

Built-in admission controllers you have already met

CONTROLLER	DOES
NamespaceLifecycle	Refuses new objects in a namespace being deleted
LimitRanger	Applies LimitRange defaults and limits (module 11)
ResourceQuota	Enforces namespace quotas (module 11)
ServiceAccount	Sets the default ServiceAccount and token (module 17)
PodSecurity	Applies the Pod Security Standards
DefaultStorageClass	Adds the default StorageClass to claims (module 20)

ON THE EXAM

Two more built-ins call out to webhooks: MutatingAdmissionWebhook and ValidatingAdmissionWebhook.

Built-in policies versus policy engines

BUILT-IN ADMISSION POLICIES

CEL, in the API server

- ValidatingAdmissionPolicy, stable since 1.30
- MutatingAdmissionPolicy, stable in 1.36
- Rules as CEL expressions
- No webhook to run or keep available

POLICY ENGINES VIA WEBHOOKS

OPA Gatekeeper, Kyverno

- Run as webhooks beside the cluster
- Richer: generate resources, verify images
- Reports and policy libraries
- One more service to operate

THE TIEBREAKER

Simple validation or mutation: the built-in policies. Generating resources or verifying signatures: an engine.

Policy tools you should recognise

TOOL	KNOWN FOR
`OPA Gatekeeper`	Open Policy Agent for Kubernetes; policies in the Rego language. OPA is CNCF graduated
`Kyverno`	Policies written as Kubernetes YAML; validate, mutate, generate, verify images. CNCF graduated 2026
`ValidatingAdmissionPolicy`	Built in; CEL expressions; nothing extra to install

> ON THE EXAM
>
> Rego is OPA's language. Kyverno needs no new language: its policies are YAML.

> Quayside wants to block privileged containers, host namespaces and hostPath volumes in its application namespaces.
>
> Typical application Pods must keep working unchanged, and the team wants a built-in feature.

What should they do?

A Label the namespaces pod-security enforce: baseline

B Label the namespaces pod-security enforce: restricted

C Create PodSecurityPolicy objects and bind them

D Apply a default-deny NetworkPolicy to each namespace

◆ PAUSE AND DECIDE

What should they do?

A Label the namespaces pod-security enforce: baseline

Blocks exactly those settings; ordinary apps pass unchanged.

B Label the namespaces pod-security enforce: restricted

Also blocks them, but many unchanged apps would fail.

C Create PodSecurityPolicy objects and bind them

Removed in 1.25.

D Apply a default-deny NetworkPolicy to each namespace

Controls traffic, not what a Pod may do to its node.

THE SIGNAL

Block known escalations, keep typical apps working: Baseline.

> The payments namespace enforces Baseline. The team wants to move it to Restricted next quarter.
>
> First they want to know which workloads would break, without blocking anything today.

What should they do now?

A Switch enforce to restricted over a quiet weekend

B Set every namespace to the Privileged level for now

C Keep enforce baseline; add warn and audit at restricted

D Delete any Pod that does not look compliant

◆ PAUSE AND DECIDE

What should they do now?

A Switch enforce to restricted over a quiet weekend

Blocks things today, which is exactly what they must not do.

B Set every namespace to the Privileged level for now

Removes protection and reveals nothing.

C Keep enforce baseline; add warn and audit at restricted

Reports every future violation, blocks nothing new.

D Delete any Pod that does not look compliant

Guesswork, and causes outages.

THE SIGNAL

'Find out what would break, block nothing': warn and audit at the stricter level.

> The quayside namespace now enforces Restricted. A new Deployment is created without error, but it never gets any Pods.
>
> The ReplicaSet's events say: violates PodSecurity restricted: allowPrivilegeEscalation != false.

What explains this, and what is the fix?

A. The Deployment was rejected; re-create it with --force

B. RBAC forbids the ReplicaSet from creating Pods

C. The namespace ResourceQuota has run out of Pods

D. Pods are rejected; fix the template's securityContext

◆ PAUSE AND DECIDE

What explains this, and what is the fix?

A The Deployment was rejected; re-create it with --force

It was accepted. Only its Pods are rejected.

B RBAC forbids the ReplicaSet from creating Pods

The event names PodSecurity, not a permission.

C The namespace ResourceQuota has run out of Pods

A quota event would say exceeded quota.

D Pods are rejected; fix the template's securityContext

Set allowPrivilegeEscalation: false, and the rest of Restricted.

THE SIGNAL

Deployment accepted, zero Pods, PodSecurity in the events: fix the Pod template.

> Quayside wants two organisation-wide rules. Only images from registry.quayside.example may run.
>
> And every new namespace must automatically get a default NetworkPolicy created inside it.

Which tool fits both rules?

A Pod Security Admission at the Restricted level

B A policy engine such as Kyverno

C RBAC roles limiting who may create Pods

D NetworkPolicies in every namespace

◆ PAUSE AND DECIDE

Which tool fits both rules?

A Pod Security Admission at the Restricted level

Checks the fixed standards only; knows nothing of registries.

B A policy engine such as Kyverno

Validates image sources and generates resources in new namespaces.

C RBAC roles limiting who may create Pods

Controls who, not which images or what gets created.

D NetworkPolicies in every namespace

The thing to be generated, not the tool that generates it.

THE SIGNAL

Custom rules plus generating resources: a policy engine.

Module 18 in five lines

- Three standards: **Privileged, Baseline, Restricted**.
- **Pod Security Admission** replaced PodSecurityPolicy in 1.25. Namespace labels pick the level.
- Modes: **enforce** rejects, **audit** logs, **warn** warns. Enforce checks Pods, not Deployments.
- Restricted needs **non-root, no escalation, drop ALL, seccomp**. User namespaces stable in 1.36.
- Custom rules: **ValidatingAdmissionPolicy** (CEL) built in; **Gatekeeper** and **Kyverno** go further.

Pods are locked down. Next: **locking down the network between them**, and the supply chain behind them.